

Inferential Statistical Analyses

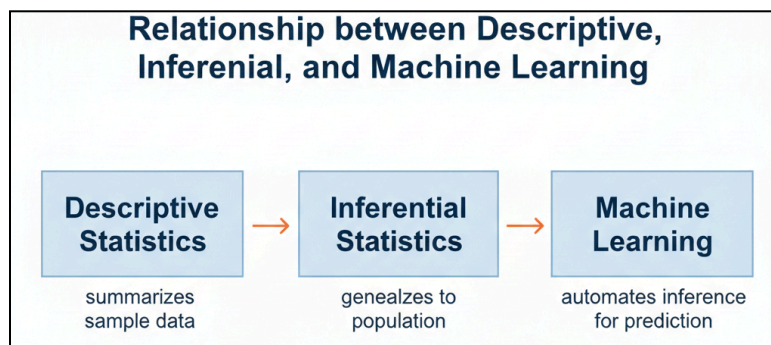
Introduction

Inferential statistical analysis is a branch of statistics that allows conclusions to be drawn about a population based on data collected from a sample. It provides tools to make predictions, test hypotheses, and estimate relationships between variables.

Unlike descriptive statistics, which only summarize data, inferential statistics use **probability theory** to infer patterns and make generalizations beyond the observed dataset.

In simple terms, descriptive statistics tells us **what has happened**, while inferential statistics tells us **what could happen** or **what is true for the population**.

In the context of data science, inferential statistics form the foundation of **machine learning algorithms**, where models are trained on samples and expected to perform accurately on unseen data.



1. Machine Learning vs Descriptive Statistics

Descriptive statistics and machine learning differ in **purpose**, **depth**, and **analytical goals**.

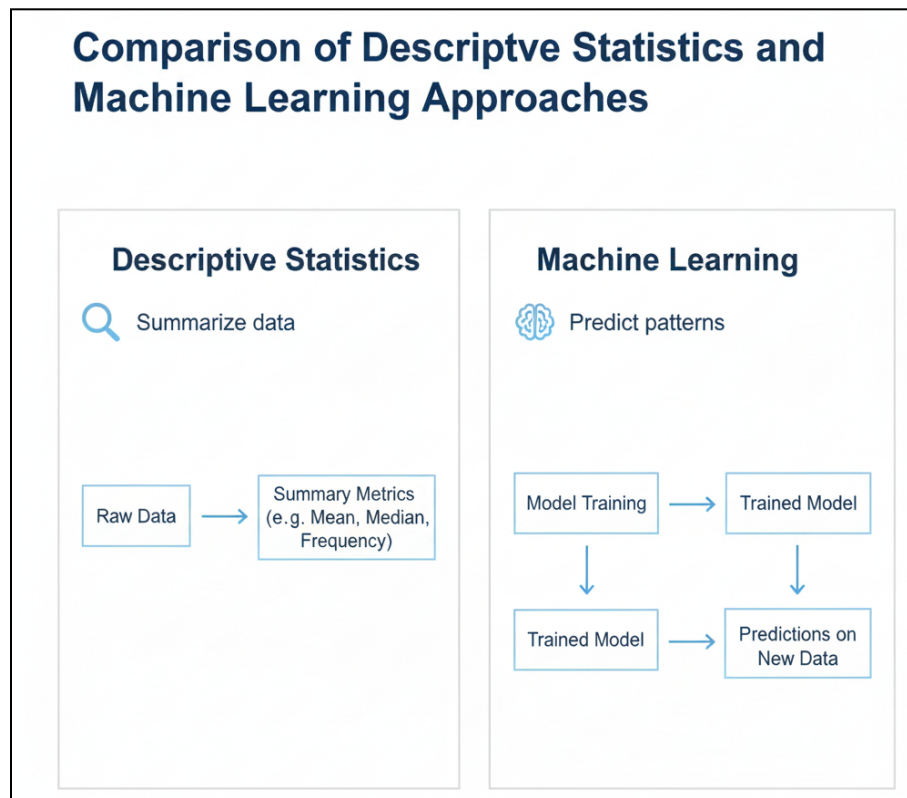
Descriptive statistics aim to **summarize and describe existing data**, while machine learning — grounded in **inferential reasoning** — focuses on **discovering patterns and making predictions** about future or unseen data.

Aspect	Descriptive Statistics	Machine Learning (Inferential Approach)
Goal	Summarize existing data	Learn patterns and make predictions
Data Type	Observed and complete dataset	Training data, generalized to unseen data
Nature of Analysis	Retrospective (past data)	Predictive (future or unknown data)
Mathematical Focus	Measures of central tendency, dispersion, and correlation	Optimization, estimation, and model training
Common Techniques	Mean, Median, Mode, Variance, Standard Deviation	Regression, Classification, Clustering, Neural Networks
Result	Reports and visual summaries	Predictive or decision-making models

Example:

- **Descriptive:** “The average monthly income of users is \$3,500.”
- **Machine Learning:** “Predict whether a new customer will earn above \$3,500 based on age, education, and location.”

Descriptive statistics help in **understanding patterns within observed data**, whereas machine learning — based on inferential principles — helps in **uncovering hidden relationships and making reliable predictions**.



Inferential statistical analysis acts as the **bridge between these two**, ensuring that patterns derived from a sample are statistically valid for the broader population.

1.1. Hypothesis Testing

Hypothesis testing is a formal statistical procedure used to **compare observed data with expectations** derived from a specific assumption, known as the **null hypothesis (H_0)**.

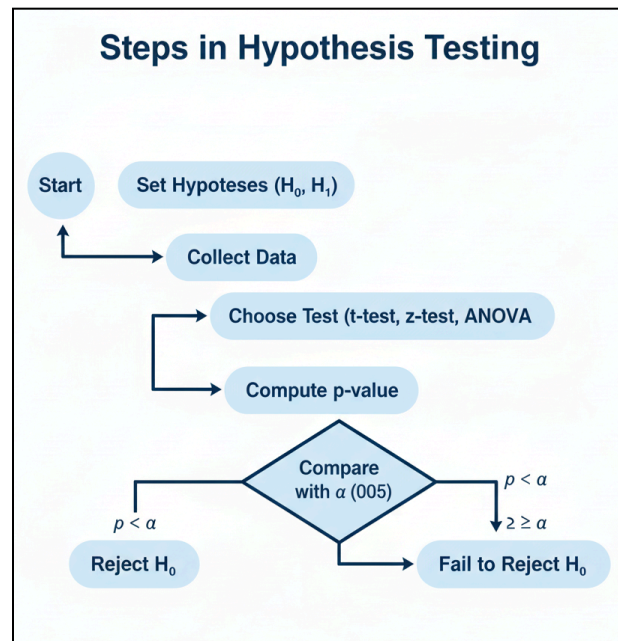
It allows researchers to determine whether an observed effect or difference is statistically significant.

- **Null Hypothesis (H_0):** There is no significant difference or effect.
- **Alternative Hypothesis (H_1):** There is a significant difference or effect.

Example:

- H_0 : The average exam score = 70
- H_1 : The average exam score $\neq$ 70

Data are collected and analyzed using inferential tests such as the **t-test**, **z-test**, or **ANOVA** to decide whether to reject the null hypothesis.



p-value

The **p-value** indicates the probability of obtaining the observed results (or more extreme) if the null hypothesis were true.

Condition	Interpretation
$p < 0.05$	Reject $H_0 \rightarrow$ result is statistically significant
$p \geq 0.05$	Fail to reject $H_0 \rightarrow$ result not statistically significant

Example in Python:

```
from scipy import stats
```

```
data = [67, 70, 68, 72, 66, 74]
t_stat, p_val = stats.ttest_1samp(data, 70)
print("t-statistic:", t_stat, "p-value:", p_val)
```

Explanation:

The test compares the sample mean with the hypothesized population mean (70). If the p-value is less than 0.05, we reject H_0 and conclude that the average score differs significantly from 70.

1.2. Confidence Intervals

A **confidence interval (CI)** provides a range of values that likely contains the true population parameter (such as the mean or proportion).

It expresses the degree of uncertainty associated with a sample estimate.

Interpretation:

For example, a **95% confidence interval** for the mean = 68 ± 2 indicates that we are **95% confident** that the true population mean lies between **66 and 70**.

- A **wider interval** indicates more uncertainty (smaller sample size or higher variability).
- A **narrower interval** implies more precision (larger sample size or lower variability).

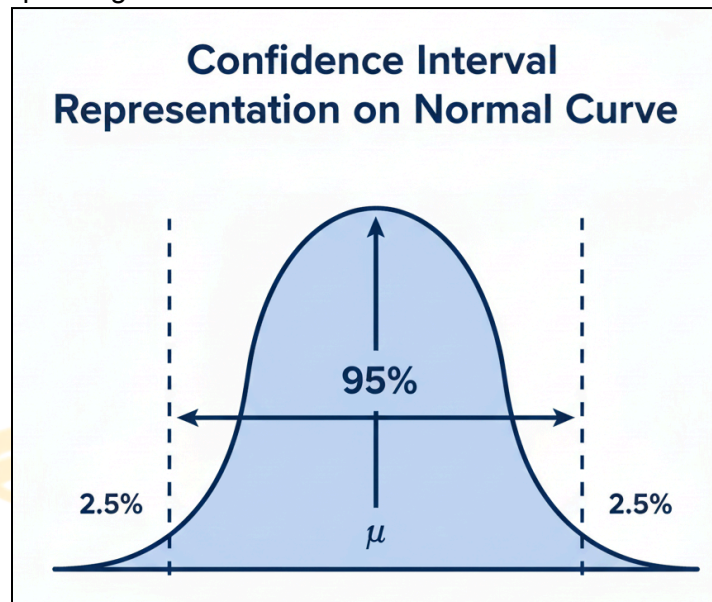
- Common confidence levels are **90%, 95%, and 99%**.

Formula (for population mean, when σ known):

$$CI = \bar{x} \pm z \times \frac{\sigma}{\sqrt{n}}$$

where,

- $\bar{x}$ = sample mean,
- σ = population standard deviation,
- n = sample size,
- z = z-score corresponding to the desired confidence level.



Example in Python:

```
import numpy as np
import scipy.stats as stats

data = [67, 70, 68, 72, 66, 74]
mean = np.mean(data)
confidence = 0.95
interval = stats.t.interval(confidence, len(data)-1, loc=mean, scale=stats.sem(data))
print("95% Confidence Interval:", interval)
```

Explanation:

This code computes the 95% confidence interval for the sample mean, showing the range within which the true population mean is likely to fall.

2. scikit-learn Toolkit

Scikit-learn (sklearn) is a powerful Python library designed for implementing inferential and machine learning models.

It provides tools for data preprocessing, model training, testing, and evaluation.

It supports both **supervised** (predictive) and **unsupervised** (pattern discovery) learning.

Key Features:

1. **Simple and Consistent API:** Easy to integrate with NumPy, Pandas, and Matplotlib.
2. **Wide Range of Models:** Includes regression, classification, clustering, and dimensionality reduction.
3. **Model Selection Tools:** Supports cross-validation, grid search, and performance evaluation.
4. **Preprocessing Utilities:** Scaling, encoding, normalization, and feature extraction.

Commonly Used Modules in Scikit-learn:

Module	Purpose
<code>sklearn.linear_model</code>	Regression and classification models
<code>sklearn.cluster</code>	Clustering algorithms (K-Means, DBSCAN, etc.)
<code>sklearn.preprocessing</code>	Data cleaning and transformation
<code>sklearn.model_selection</code>	Cross-validation and train-test splitting
<code>sklearn.metrics</code>	Performance evaluation metrics

Example Code:

```

from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# Sample data
X = [[1], [2], [3], [4]]
y = [2, 4, 6, 8]

# Split into training and testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict and evaluate
predictions = model.predict(X_test)
print("Mean Squared Error:", mean_squared_error(y_test, predictions))

```

Explanation:

The above code demonstrates how inferential models in scikit-learn use data samples to infer relationships (here, a linear one) and predict outcomes on unseen data.

3. Data Dimensionality

Data dimensionality refers to the number of features (variables) used to describe a dataset.

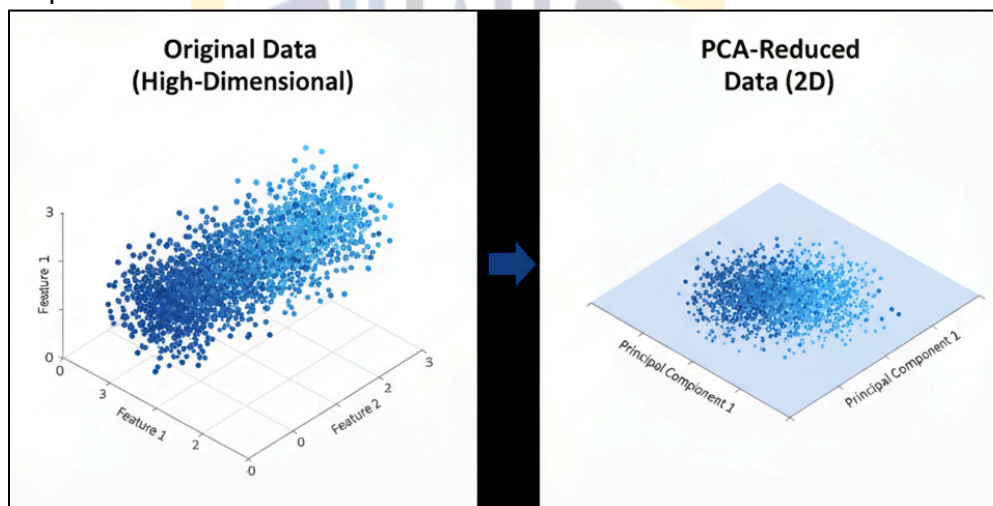
In inferential statistics and machine learning, **high-dimensional data** can lead to problems such as overfitting and increased computational cost.

Key Concepts:

1. **High Dimensionality:** When the number of features is very large compared to observations (e.g., 10,000 features with 500 samples).
2. **Curse of Dimensionality:** Model performance decreases as feature space grows because data becomes sparse.
3. **Dimensionality Reduction:** The process of transforming data into a smaller set of features that retain essential information.

Common Dimensionality Reduction Techniques:

- **Principal Component Analysis (PCA):** Converts correlated features into a smaller number of uncorrelated components.
- **Linear Discriminant Analysis (LDA):** Reduces dimensions while maximizing class separability.
- **Feature Selection:** Removes irrelevant or redundant variables using statistical tests or model importance scores.

**Example (PCA in Scikit-learn):**

```
from sklearn.decomposition import PCA
from sklearn.datasets import load_iris
```

```
data = load_iris()
pca = PCA(n_components=2)
reduced_data = pca.fit_transform(data.data)
```

Explanation:

PCA reduces the original four-dimensional Iris dataset to two dimensions while maintaining most of the variance.

4. Clustering (Unsupervised Learning)

In inferential terms, these validation indices estimate how well inferred cluster structures generalize beyond the observed data, ensuring the clustering captures true underlying distributions rather than random patterns.

Clustering is an **unsupervised learning technique** used to group similar data points into clusters without predefined labels. It helps identify hidden patterns or natural groupings within data.



4.1 Types of Clustering Algorithms

- K-Means Clustering**
 Chooses K clusters; each point belongs to the cluster with the nearest centroid. It minimizes within-cluster sum of squares (inertia).
- DBSCAN (Density-Based Spatial Clustering of Applications with Noise)**
 Clusters based on density: points that are closely packed together form clusters, while low-density points are considered outliers.
- Hierarchical Clustering (Agglomerative / Divisive)**
 Builds a tree of clusters: either bottom-up (agglomerative) or top-down (divisive). Different linkage metrics (single, complete, average) and distance metrics affect results.

Example: K-Means Clustering

```
from sklearn.cluster import KMeans
import numpy as np
# Sample data
X = np.array([[1,2], [1,4], [1,0],
              [10,2], [10,4], [10,0]])

kmeans = KMeans(n_clusters=2)
kmeans.fit(X)

print("Cluster Centers:", kmeans.cluster_centers_)
print("Labels:", kmeans.labels_)
```

Explanation:

K-Means divides data into two clusters, learning from data points' distances.

The algorithm infers the best cluster centers — representing inference in an unsupervised context.

5. Evaluating Clusters (Clustering Validation)

Since clustering is an **unsupervised learning** technique (no predefined class labels), evaluating the quality of clusters is more complex than in supervised learning.

Cluster validation aims to determine how well the data has been grouped, whether clusters are meaningful, and how many clusters best represent the underlying structure.

The evaluation mainly relies on **internal validation metrics**, which use intrinsic properties of the data itself.

5.1 Internal Validation Metrics

These metrics assess the quality of clustering using compactness (within-cluster similarity) and separation (between-cluster distance).

Some commonly used metrics include:

1. Silhouette Score:

Measures how similar a point is to its own cluster compared to other clusters.

- Value ranges from **-1 to +1**.
- Higher values indicate better-defined clusters.

2. Davies–Bouldin Index:

Represents the ratio of within-cluster scatter to between-cluster separation.

- **Lower values** indicate more compact and well-separated clusters.

3. Dunn Index:

Ratio of the minimum inter-cluster distance to the maximum intra-cluster distance.

- **Higher values** signify better clustering.

4. Calinski–Harabasz Index:

Measures the ratio of between-cluster variance to within-cluster variance.

- **Higher values** indicate well-separated and compact clusters.

5. Inertia (Within-Cluster Sum of Squares):

Measures the compactness of clusters by summing squared distances of points from their respective cluster centers.

- **Lower values** indicate tighter, more coherent clusters.
- Commonly used with the **Elbow Method** to determine the optimal number of clusters.

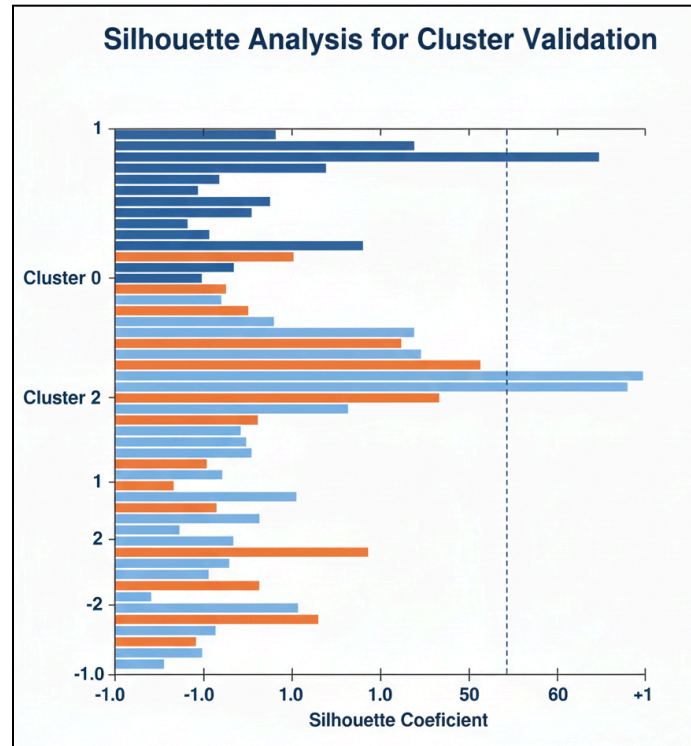
Example (Silhouette Score in scikit-learn):

```
from sklearn.metrics import silhouette_score
```

```
score = silhouette_score(X, kmeans.labels_)  
print("Silhouette Score:", score)
```

Interpretation:

A **high silhouette score** means the clusters are well separated and internally cohesive.



5.2 Methods to Determine the Optimal Number of Clusters

Since the number of clusters (K) is often unknown, several empirical methods are used to determine the most appropriate value.

1. Elbow Method:

Plot the **inertia (within-cluster sum of squares)** against the number of clusters (K).

The point where the curve bends (the “elbow”) indicates the optimal K, beyond which improvement slows significantly.

2. Silhouette Analysis:

Compute the average silhouette score for different K values and select the K that yields the **highest score**.

3. Gap Statistic:

Compares total within-cluster variation for different K values with reference datasets.

Less commonly implemented in standard scikit-learn workflows but conceptually useful.

External Validation:

When true class labels are available, clustering performance can be compared to them using metrics like Adjusted Rand Index (ARI) or Normalized Mutual Information (NMI).